Using embeddings to understand unstructured data

So far, we've focused on how to explore your structured data. What about unstructured data, such as images or text? Recall that we converted PDF text chunks into a specific format called embeddings in *Chapter 3*'s RAG chatbot project work. We require embeddings, meaning numerical vector representations of the data, to perform a similarity (or hybrid) search between chunks of text. That way, when someone asks our chatbot a question, such as "What are the economic impacts of automation technologies using LLMs?" the chatbot will be able to search through the stored chunks of text from the arXiv articles, retrieve the most relevant chunks, and use those to better answer the question. For more visual readers, see the data preparation workflow in *Figure 4.14*. We completed the **Data Preparation** step in *Chapter 3*. We'll run through the remaining setup steps in the workflow now.

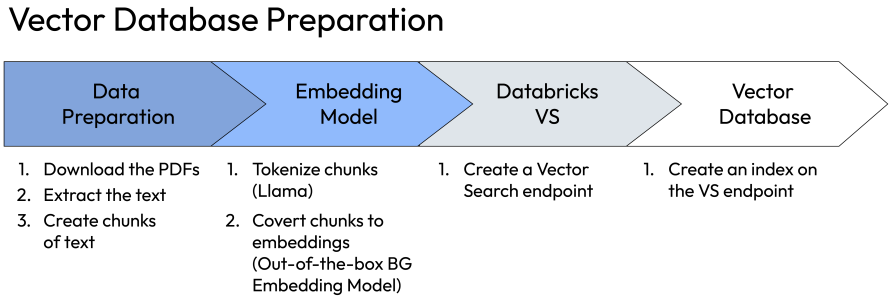


Figure 4.14 – Vector database setup is the prerequisite process supporting RAG's retrieval step

Embeddings are an essential part of building any chatbot. Pay attention to your embedding model to ensure it is relevant to the task. You wouldn't want to build a chatbot designed to answer questions in French but use an embedding model that only knows English – your chatbot's response quality will definitely suffer!

Embeddings that capture the nuances of language are essential for the chatbot to understand and generate contextually relevant responses. Equally important are the searching and filtering techniques you apply to the **vector database** itself. A vector database is similar to a SQL database, but instead of storing tabular data, it stores vector embeddings. A search algorithm can then search the embeddings. In the final flow of a RAG project, a user's question is also converted into embeddings, and the search algorithm uses those embeddings to find similar embeddings stored in the vector database. The chatbot receives the most similar embeddings from the vector database to help it craft a response to the user's question.

Let's consider the requirements of a good vector database solution:

- **Quality of the retrievals**: The correctness and completeness of embeddings returned as relevant by the search algorithm
- **Scalability of the solution**: The ability to scale per the number of requests coming to the application with dynamic traffic